

Using the platform

For everyone at iHelp — technical or not. Read once, then keep it open for the first week.

<https://ihelp-ops-live.vercel.app>

The one thing to understand

Nobody types how far along they are.

There is no "80% done" box anywhere in this platform, and there never will be. A task moves because something real happened in GitHub: a branch appeared, a commit landed, a pull request opened, a person approved it, it merged.

That means two things for you.

You cannot fall behind by forgetting to update a status. If you did the work, the platform already knows. You never have to report progress.

You also cannot get ahead by saying you did. Nobody can, including the founder. That is the point — everyone is measured the same way, by what exists.

Getting in

1. Go to the URL above and press **Sign in with Google**. Use your work Gmail.
2. That's it. Your account is created the first time you sign in.

Linking GitHub

If you write code, you also need a GitHub account linked to you. Ask for it to be set up once — it is a one-line change by whoever runs the platform.

Why it matters: git signs every commit with a GitHub username. Reviews, approvals and code ownership all use it. An account with no GitHub username attached cannot *own* work, so the platform will not let it start a task or run an agent. This is deliberate, not a bug.

Without GitHub linked you can still: read everything, book leave, comment on tasks, see your own goals and feedback — and, if you are a lead, assign work to other people. Only *doing* the work is closed to you: taking a task, starting one, and dispatching an agent.

The screens

SCREEN	WHAT IT'S FOR
Board	Every open task across every repository, and how far each has really got
Team	Who's on which pod, what they have open, who's away today
Leave	Book time off, see your balance, approve your team's requests
Me	Your goals, your 1:1 notes, feedback about you, what you've shipped
Analytics	How work is flowing across the whole team
Admin	Accounts, roles and agent tiers. Only the CTO and the founder see this

The board covers **every repository the company has**, including ones created after you read this — it reads the list from GitHub rather than a setting somebody has to remember to update. Tasks are grouped under the repository they belong to.

Repositories with nothing open aren't listed; they're counted on one line at the bottom. Thirteen headings with "nothing here" under eleven of them buries the two tasks that matter.

That's also why a task's address has the repository in it —

`/task/ihelprobotics/ihelp-ops/42` . Issue numbers restart in every repository, so "#1" on its own doesn't say which piece of work you mean.

Roles

Four of them. Your role decides what you may do to **other people's** work and records — never what you may do to your own.

ROLE	WHAT IT ADDS
member	Take work, do it, run agents up to your tier, book your own leave. Where everybody starts
lead	Plus: assign work to people, approve their leave, see their leave and nudges
cto	Plus: all of that for everybody — and the only role that can read a 1:1 it did not write
founder	A lead's reach across everybody, and any agent. Deliberately <i>not</i> able to read other people's 1:1 notes

Separately, an **agent tier** — `week1` , `week2` or `full` — decides which agents you can dispatch. It grows as you learn what good looks like here.

Neither your role nor your tier is a judgement. Both are a starting point.

The CTO reading a 1:1 is logged, and you see the log. It is on your own page under "who has read notes about you". That is what makes "the CTO can read everything" something you can live with rather than something you have to trust.

Who changes a role

The CTO and the founder, on the **Admin** screen — which only they can see. Nobody changes their own row, including them: a role change should be something a second person agreed to. The full table of what each role can do lives on that screen, and every line names where it is enforced.

Someone new signs in with Google and appears there as a `member` on `week1` with no GitHub login. Setting that login is what turns an account

into somebody who can own work.

Every role change is recorded — who made it, when, and what it was before. You can see the changes to your own account on your own page, and nobody can edit or remove a line, including an admin. Your role changing is a thing that happened to you, so it is a thing you can read.

Getting a task

Every task has **exactly one owner, or nobody**. Never two. A task with two owners has none — "somebody was going to do it" is the outcome this whole system exists to prevent.

There are two ways your name gets on one, and both are normal.

But first somebody has to open it.

Opening a task

New task in the header. Pick the repository, write a title, write a description, and press **Open the task**. It becomes a GitHub issue and you land on its task page at 10%.

Two things to know. Your GitHub account must be linked — a task is a GitHub issue, and who holds it, whose commits count and who may approve it are all keyed on a username. And the description is not a note, it is the brief: an agent dispatched onto this task is given that text and nothing else. Three clear sentences take less time to write than reviewing the confident, wrong pull request that a vague one produces.

You can also open one straight on GitHub — `gh issue create` or the New issue button — and it appears here just the same. The platform reads tasks live; it never keeps its own copy.

You take it

Find something on the board with **nobody has taken this** under it and press **Take it**. That's the whole thing — no permission, no waiting. If you can see the work, you can start it.

You can also take it from the task's own page, under **Owner**.

You can't take a task somebody else already holds. The button won't be there, and the platform will tell you who has it. Go and ask them — they

might be stuck, or finished, or glad to hand it over. If they agree, they press **Hand it back** and you take it, or a lead moves it directly.

Somebody gives it to you

Leads, the CTO and the founder can assign anyone. On the task page they get **Assign to...** and can pick a person, or **Reassign** to move it off whoever has it now.

That last part matters: work can be moved off someone who's stuck, on long leave, or has left. Without it a task would belong to whoever touched it first, for ever.

You don't need a GitHub account to assign work. The platform assigns using its own credentials, so a delivery manager who has never written a line of code can hand work out all day. Only *doing* the work — branches, commits, pull requests — needs a linked GitHub account, because those things get signed with a git identity.

Giving one back

If it's yours and you haven't started, press **Hand it back**. It returns to the board with nobody's name on it. No explanation owed — better an honest hand-back than a task sitting under your name for three weeks.

How a task actually moves

Seven steps. Each one is caused by a real thing, never by clicking a button that says "done".

BAR READS WHAT MADE IT MOVE

10%	Somebody opened the issue on GitHub — it may not have an owner yet
20%	A branch was created for it
40%	The first commit landed on that branch
60%	A pull request was opened
75%	The automated checks passed
90%	A human approved it
100%	It was merged

If a task is stuck at 10%, nothing has happened yet. Not "somebody forgot to update it" — genuinely nothing. That is useful information rather than a failure of admin.

Doing a piece of work

1. Make sure it's yours

Take it from the board, or check the **Owner** section on the task page says your name. Everything below assumes the task is yours — that's what makes the work attributable to you rather than to nobody.

2. Start it

Press **Start task**. The platform creates a branch named after the issue — `task/42/fall-detection-threshold`. The number in the branch name is how everything you do afterwards attaches itself to the task automatically.

3. Open it in VS Code

The task page gives you three ways in:

- **Desktop** — opens VS Code on your machine
- **Browser** — opens the editor in a browser tab, nothing to install
- **Terminal** — the `git` command, if you already have the repo cloned

4. Set up the commit hook — once per clone

```
git config core.hooksPath .githooks
```

Do this once, the first time you clone the repository. It makes every commit tag itself with the task number, which is what puts your work on the board without you doing anything else.

If you skip it, your commits still land — they just don't attach to any task, and the board won't move.

5. Work, commit, push

Normal git. Commit and push as often as you like. Within a minute of pushing, the task moves to 40% on its own.

6. Open a pull request, get it reviewed, merge

You can do all three from the task page. The **Merge** button calls GitHub — it does not override anything. If GitHub refuses because a review is missing, that refusal is the protection working, not a broken button.

Using an agent

Agents are AI teammates. There are two ways to work with one, and the difference between them is the whole point.

Talk to it — the conversation on every task page. It reads, thinks and answers in seconds, and changes nothing.

Run it — the **Run** button. It goes to work in GitHub Actions and a few minutes later there's a pull request waiting for you.

The fast thing is cheap. The consequential thing is evidenced. Use the first to work out what to do, and the second to do it.

Talking to one

Open a task, scroll to **Ask an agent**, pick who you want and type. The answer streams back as it's written.

The agent has read the task, its comments, and its own brief from the repository — so the QA agent you're talking to holds the same standards as the QA agent that opens the pull request. Ask it what a task actually needs, how the code around it works, whether your approach is sound, or what it would need to read before it could tell you. If it doesn't know, it says so and names what it would need. That is a better answer than a confident wrong one.

It cannot change anything. No edits, no commits, no pull requests — those tools don't exist in the conversation. If it ever describes an edit as though it made one, it is wrong and you should not believe it. To get code changed, press **Run**.

The conversation is yours. Nobody else can read it — not your lead, not the CTO, not the founder. There's no admit-the-boss setting; the database policy has one clause and `npm run test:chat` proves it by signing in as a founder and failing to read one.

It is not the record. Nothing you say to an agent moves a task, and nothing it says counts as evidence of anything. If something from the conversation matters — a decision, a risk, a reason — put it on the issue as a comment. That is public, it's on GitHub, and it's what the next person will read.

You can talk to all eight agents from your first day, whatever your tier: the tiers below gate **Run**, not conversation. Each answer shows what it cost, usually a fraction of a cent.

You can delete a conversation. You cannot edit one after the fact — not even your own words. A transcript you can rewrite is one nobody can rely on, including you.

Running one

What you can run depends on your tier

TIER	AGENTS YOU CAN DISPATCH
week1	Scribe (docs), Data Annotator, QA
week2	the above, plus Frontend
full	the above, plus Full-stack, AI Developer, Integrator, Architect

Locked agents stay visible and greyed out, with the reason showing. That's on purpose — a hidden button teaches you nothing.

Some agents — Deployer, Cloud, Social, Outreach, Lead Gen — can never be dispatched from here. Their human owner runs them.

Two rules about agents

You own what the agent produced. Your name goes on the pull request. Read every line, run it, and be ready to explain it. "The agent wrote it" is not an answer anyone will accept, and the platform records who asked for the run.

Two runs at a time, each. Same work-in-progress limit as everything else.

Every run records what it cost. You can see it on the task and on Analytics.

Booking leave

Go to **Leave**, pick the kind, the dates, and press **Request**.

KIND	COMES OFF YOUR ANNUAL BALANCE?
Planned	Yes
Unpaid	Yes
Sick	No
Comp off	No

Your lead approves it — or the CTO, if you have no lead set. You'll see who it's waiting on. You can withdraw a request while it's still pending.

Why leave is in an engineering tool at all: so nobody chases you on a day you're away. Once your leave is approved, the daily email skips you entirely. That's the whole reason it exists.

A rejection always comes with a written reason. You'll see it on your own **Leave** page.

Your page

Me shows four things about you.

Goals — your 30/60/90 day goals, with the real work linked against them. They're reported as *counts of evidence*, never as a score out of ten. You will never see a percentage next to your name in this platform.

1:1 notes — what was discussed, and what was agreed.

Feedback — notes written about you.

Who has read notes about you — a log.

Who can read what

This is enforced by the database itself, not by a policy anyone can forget.

- **You** can read *everything* written about you. There are no private manager files. If it exists, you can see it.
- **The person who wrote a note** can read their own note.
- **The CTO** can read all notes — and every time they read one they didn't write and isn't about them, it appears in your access log.
- **Nobody else** can read notes about you. Not your lead, not another lead, not a colleague.

If a colleague opens your page and sees nothing under 1:1s, the page tells them *why* it's empty rather than pretending you have no record.

Your conversations with agents

Stricter than notes: **nobody but you**. Not your lead, not the CTO, not the founder — there is no equivalent of the CTO's read-all-notes power here, and no access log because there is no access to log.

That is the trade that makes storing them acceptable at all. This is the only place in the platform that keeps what somebody typed; `local_session` records that an editor session happened and deliberately not a word of what was in it. A conversation can only be kept because a conversation with no history is not a conversation — and because it is yours, it is not evidence of anything, and you can delete it.

Analytics

Team-wide, not personal. The numbers worth knowing:

Cycle time — how long a task took from being opened to being merged.

Review wait — how long a pull request sat before a human approved it. If this climbs, reviews are the bottleneck, not the coding.

Claimed, and proven — open tasks with somebody's name on them and no branch, commit or pull request behind them yet. This is not a naughty list. A task can be assigned this morning, or be a week of reading first. It's simply the one place where what's claimed and what exists sit side by side.

Stalled — nothing has happened on it for three days or more. A prompt to ask, not a verdict.

Cost — what the agents spent, per agent and per person.

There are no hours anywhere. No time online, no time tracked, no presence. The moment those sit next to real work, people optimise the easier number. Cycle time measures how long the *work* waited — not how long you sat at a desk.

The daily email

At 6pm IST, two things go out.

A **nudge**, only to someone with nothing recorded that day and no blocker raised. It is a prompt, not a telling-off — plenty of days are reading, debugging or meetings. You get at most one a day, ever. And if you're on approved leave, you get none.

A **digest** to the CTO and delivery manager, covering everybody in one email.

If you're stuck

Raise it in writing — label the issue `blocked` , or say so in your pod channel.

> A block that is raised belongs to the company. A block that is not raised > belongs to you.

That's the deal. Nobody minds a blocker. Silence is the only thing that causes problems.

If something looks wrong

Every error in this platform tells you what failed and what to check. If you see one that doesn't, that's a bug worth reporting.

A few things that look like bugs but aren't:

"Every task says 10%" — no GitHub events have reached the platform. The board says so explicitly rather than showing you a quiet week.

"The Merge button refused" — branch protection. A required review or check is missing. Working as intended.

"My commits aren't showing" — the commit hook probably isn't installed in that clone. Run `git config core.hooksPath .githooks` and push again.

"I can't run any agents" — your GitHub account isn't linked yet, or the agent is above your tier. The tile says which.

"There's no Take it button" — either the task already has an owner (it says who), or your GitHub account isn't linked yet.

"The same number appears twice" — they're different tasks in different repositories. Issue numbers restart in each one, which is why the repository name is in every link.

If you don't write code

You can do all of this:

- Read every task across every repository, and see exactly where each stands
- Comment on tasks — your comment goes to the real GitHub issue
- Book and track leave
- See your goals, your 1:1 notes and feedback about you
- See the team and who's away

And if you're a lead, the CTO or the founder, you can assign work — hand a task to somebody, or move one off a person who's stuck or away. That needs no GitHub account at all, because the platform assigns using its own credentials.

What you can't do is *start* a task or run an agent. Those write commits and pull requests, which get signed with a git identity, and an account without one can't own them. That's a limit on how git works, not a comment on you.